

Migração frontend em camadas (SPA)

Paridade arquitetural com o backend no critério do client:
barrel único, serviços por capacidade, páginas finas e invalidação escopada.

React · Vite · TanStack Query · services/api · sem DI container

Setembro 2026

Critério de 100% (frontend)

O transporte HTTP do SPA já estava centralizado. Esta migração fechou a paridade de organização: fachada de API, split de serviços misturados, extração de páginas gordas e fim da invalidação global de cache.

1. Por que não é o mesmo stack do backend

No Django migrámos regra de negócio para domínio/application com DI. No React, a regra continua no backend. O SPA adapta contratos HTTP e orquestra UI. Portar lifetimes SCOPED/TRANSIENT ou um container Inversify atritaria o modelo do TanStack Query sem ganho. Composition root = AppProviders + objetos *Api.

pages/components → services/api.ts → domain/*Api → infra/http.ts
lib/ = helpers puros · hooks/ = ciclo de UI sem paths de API

2. O que mudou

Frente	Resultado
Barrel api.ts	Exporta programs, commerce, catalog, staffGameContent, financialReports, customItems, itemObservation.
sendJson	Helper compartilhado; commerce não importa programs.
Catálogo	HTTP em catalogApi; index/search em lib/; hook useItemCatalog.
Split programs	programsApi = produto; gamesApi = gameplay + details; staffGameContentApi = CRUD staff.
Páginas finas	Inventory, Rankings, Wallet, Rewards, Auction, Marketplace, Help, admins.
Invalidação	useProgramAction exige queryKeys; sem invalidateQueries() global.

3. Decisões e por quê

Casa única para games

Battle pass/daily/fishing details saíram de programsApi. Um só lugar evita drift entre RewardsPage, FishingGame e GamesPage.

Sem DI container

Providers React + imports explícitos cobrem testes (vi.mock do módulo domain ainda intercepta re-exports). Um container adicionaria cerimônia sem fronteira nova.

Invalidação escopada

invalidateQueries() sem filtro revalidava o cache inteiro após cada ação de programa. Agora cada run declara as chaves (ex.: fishing + fishing-details).

4. Como implementar daqui pra frente

- Tipos sem coerção monetária imprecisa.
- Operação no domain *Api + export em services/api.ts.
- Página importa de services/api; Query + invalidação escopada.
- UI em components/<feature>; página só orquestra.
- Testes de contrato HTTP + interação Testing Library.

Checklist de auditoria

Auditoria: fetch só em http.ts; páginas sem import de domain/infra; nenhum invalidateQueries() sem queryKey; lib/ sem request.

5. Conclusão

Com o backend em Clean Architecture + DI e o frontend em camadas SPA consistentes, o caminho de feature fica simétrico: porta/use case no servidor, *Api + página fina no client. Documentação viva em docs/arquitetura/visao-geral.md e docs/desenvolvimento/frontend.md.

PDL PRO — frontend em camadas
Migrado · Validado · Pronto para novas telas